04-01 네트워킹 패키지 분석

개요

Rain BitTorrent 클라이언트의 네트워킹 계층은 피어 간 통신의 핵심을 담당합니다. 이 문서는 네트워킹 관련 주요 패키지들의 구현을 상세히 분석하여 BitTorrent 프로토콜의 네트워크 통신이 어떻게 구현되는지 설명합니다.

분석 대상 패키지

- `peer` : 피어 상태 관리와 메시지 처리
- `peerconn` : 피어 연결 추상화
- `peerprotocol` : BitTorrent 프로토콜 메시지 구현
- `acceptor` : 인커밍 연결 수락

1. peer 패키지 - 피어 상태 관리와 메시지 처리

1.1 핵심 구조체: Peer

`Peer` 구조체는 개별 피어와의 연결을 나타내며, BitTorrent 프로토콜의 모든 상태 정보를 관리합니다.

```
type Peer struct {

*peerconn.Conn



ConnectedAt time.Time
```

```
Source peersource.Source


// 피어 상태 정보
Bitfield *bitfield.Bitfield
ReceivedAllowedFast sliceset.SliceSet[piece.Piece]
SentAllowedFast sliceset.SliceSet[piece.Piece]


// 피어 식별 정보
ID [20]byte
ExtensionsEnabled  bool
FastEnabled  bool
DHTEnabled  bool
EncryptionCipher mse.CryptoMethod


// 프로토콜 상태
ClientInterested  bool
ClientChoking  bool
PeerInterested  bool
PeerChoking  bool


// 성능 관련
OptimisticUnchoked  bool
Snubbed  bool
```

```
Downloading  bool


downloadSpeed metrics.Meter
uploadSpeed metrics.Meter


// 확장 프로토콜
ExtensionHandshake *peerprotocol.ExtensionHandshakeMessage
PEX *pex


// 동시성 제어
closeC  chan  struct{}
doneC  chan  struct{}
}
```

1.2 피어 생성과 초기화

`New()` 함수는 네트워크 연결로부터 새로운 피어 객체를 생성합니다:

```
func  New(conn net.Conn, source peersource.Source, id [20]byte,
extensions [8]byte, cipher mse.CryptoMethod,
pieceReadTimeout, snubTimeout time.Duration,
maxRequestsIn  int, br, bw *ratelimit.Bucket) *Peer
```

초기화 과정:

1. **확장 기능 감지**: extensions 바이트에서 지원 기능 파싱

- Fast Extension (61번 비트)
- Extension Protocol (43번 비트)
- DHT (63번 비트)

2. **기본 상태 설정**:

- `ClientChoking = true` (초기에는 상대방을 choke)
- `PeerChoking = true` (상대방도 우리를 choke한 상태)

3. **성능 모니터링 설정**: 다운로드/업로드 속도 측정기 초기화

1.3 피어 메시지 처리 루프

`Run()` 메서드는 피어의 메인 이벤트 루프를 실행합니다:

```
func (p *Peer) Run(messages  chan Message, pieces  chan PieceMessage,
snubbed, disconnect  chan *Peer)
```

처리 흐름:

1. **메시지 수신**: `p.Conn.Messages()` 채널에서 메시지 대기
2. **메시지 분류**:

- `peerreader.Piece` : 실제 데이터 → `pieces` 채널로 전송
- 기타 프로토콜 메시지 → `messages` 채널로 전송

3. **성능 측정**: 다운로드/업로드 바이트 수 기록
4. **Snub 감지**: 타이머를 통한 느린 피어 감지

1.4 프로토콜 상태 관리

Choking/Unchoking 제어:

```go
func (p *Peer) Choke() {
p.ClientChoking = true
p.SendMessage(peerprotocol.ChokeMessage{})
}

func (p *Peer) Unchoke() {
p.ClientChoking = false
p.SendMessage(peerprotocol.UnchokeMessage{})
}
```

피스 요청:

```go
func (p *Peer) RequestPiece(index, begin, length  uint32) {
msg := peerprotocol.RequestMessage{
Index: index, Begin: begin, Length: length
}
p.SendMessage(msg)
}
```

1.5 PEX (Peer Exchange) 구현

PEX는 피어들 간에 다른 피어 정보를 교환하는 확장 프로토콜입니다.

```go
type pex struct {
```

```
conn *peerconn.Conn
extID uint8
pexList *pexlist.PEXList
pexAddPeerC chan *net.TCPAddr
pexDropPeerC chan *net.TCPAddr
}
```

PEX 동작 과정:

1. **초기화**: 기존 피어 목록으로 PEX 리스트 생성
2. **주기적 전송**: 1분마다 추가/제거된 피어 정보 전송
3. **메시지 형식**: `ExtensionPEXMessage` 로 피어 정보 교환

2. peerconn 패키지 - 피어 연결 추상화

2.1 Conn 구조체

`Conn` 은 네트워크 연결을 추상화하여 메시지 송수신 인터페이스를 제공합니다:

```
type Conn struct {
conn net.Conn
reader *peerreader.PeerReader
writer *peerwriter.PeerWriter
messages chan any
log logger.Logger
closeC chan struct{}
```

```
doneC chan struct{}
}
```

2.2 연결 생성과 설정

```
func New(conn net.Conn, l logger.Logger, pieceTimeout time.Duration,
maxRequestsIn int, fastEnabled bool, br, bw *ratelimit.Bucket) *Conn
```

구성 요소:

- **PeerReader**: 메시지 수신 담당
- **PeerWriter**: 메시지 송신 담당
- **Rate Limiting**: 대역폭 제한 (`br` , `bw` 버킷)

2.3 메시지 송수신 인터페이스

메시지 송신:

```
func (p *Conn) SendMessage(msg peerprotocol.Message) {
p.writer.SendMessage(msg)
}

func (p *Conn) SendPiece(msg peerprotocol.RequestMessage, pi io.ReaderAt) {
p.writer.SendPiece(msg, pi)
}
```

메시지 수신:

```go
func (p *Conn) Messages() <-chan any {
return p.messages
}
```

2.4 연결 관리 루프

`Run()` 메서드는 reader와 writer를 조율하여 양방향 통신을 관리합니다:

```go
func (p *Conn) Run() {
defer close(p.doneC)
defer close(p.messages)

go p.reader.Run()
go p.writer.Run()

// reader와 writer의 메시지를 통합하여 전달
for {
select {
case msg := <-p.reader.Messages():
select {
case p.messages <- msg:
case <-p.closeC:
}
case msg := <-p.writer.Messages():
```

```
select {
case  p.messages <- msg:
case <-p.closeC:
}
// 에러 처리 및 정리
}
}
}
```

3. peerprotocol 패키지 - BitTorrent 프로토콜 메시지 구현

3.1 메시지 타입 정의

BitTorrent 프로토콜의 모든 메시지 타입이 정의되어 있습니다:

```
type MessageID uint8

const (
Choke MessageID = iota
Unchoke
Interested
NotInterested
Have
Bitfield
```

```
Request

Piece

Cancel

Port

Suggest = 13

HaveAll = 14

HaveNone = 15

Reject = 16

AllowedFast = 17

Extension = 20

)
```

3.2 메시지 인터페이스

모든 메시지는 공통 인터페이스를 구현합니다:

```
type Message interface {

io.Reader

ID() MessageID

}
```

3.3 핵심 메시지 구현

Have 메시지 - 피스 보유 알림:

```go
type HaveMessage struct {
Index  uint32
}

func (m HaveMessage) Read(b []byte) (int, error) {
binary.BigEndian.PutUint32(b[0:4], m.Index)
return  4, io.EOF
}
```

Request 메시지 - 피스 요청:

```go
type RequestMessage struct {
Index, Begin, Length  uint32
}

func (m RequestMessage) Read(b []byte) (int, error) {
binary.BigEndian.PutUint32(b[0:4], m.Index)
binary.BigEndian.PutUint32(b[4:8], m.Begin)
binary.BigEndian.PutUint32(b[8:12], m.Length)
return  12, io.EOF
}
```

Bitfield 메시지 - 피스 보유 현황:

```go
type BitfieldMessage struct {
Data []byte
pos int
}

func (m *BitfieldMessage) Read(b []byte) (n int, err error) {
n = copy(b, m.Data[m.pos:])
m.pos += n
if m.pos == len(m.Data) {
err = io.EOF
}
return
}
```

3.4 확장 프로토콜 메시지

Extension Handshake:

```go
type ExtensionHandshakeMessage struct {
M map[string]uint8 `bencode:"m"`
V string `bencode:"v"`
YourIP string `bencode:"yourip,omitempty"`
MetadataSize int `bencode:"metadata_size,omitempty"`
RequestQueue int `bencode:"reqq"`
```

```
}
```

PEX 메시지:

```
type ExtensionPEXMessage struct {
Added string `bencode:"added"`
Dropped string `bencode:"dropped"`
}
```

4. acceptor 패키지 - 인커밍 연결 수락

4.1 Acceptor 구조체

```
type Acceptor struct {
listener net.Listener
newConns chan net.Conn
closeC chan struct{}
doneC chan struct{}
log logger.Logger
}
```

4.2 연결 수락 루프

`Run()` 메서드는 지속적으로 새로운 연결을 수락합니다:

```go
func (a *Acceptor) Run() {
defer close(a.doneC)

// 종료 시 리스너 닫기 위한 고루틴
done := make(chan struct{})
defer close(done)
go func() {
select {
case <-a.closeC:
a.listener.Close()
case <-done:
}
}()

for {
conn, err := a.listener.Accept()
if err != nil {
select {
case <-a.closeC:
default:
a.log.Error(err)
}
```

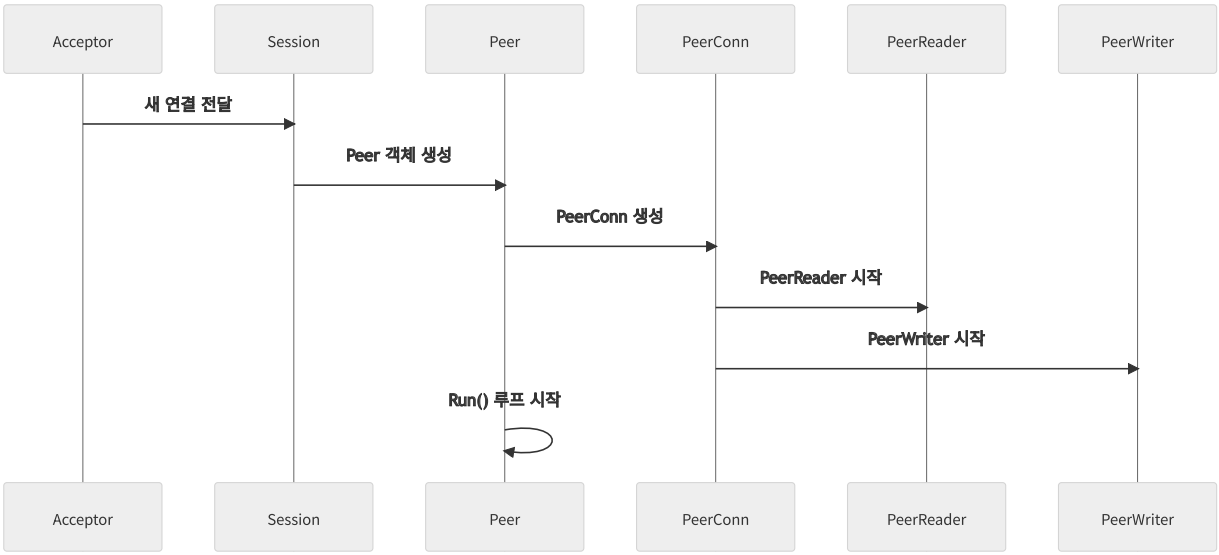
```
return
}
select {
case  a.newConns <- conn:
case <-a.closeC:
conn.Close()
return
}
}
}
```

동작 특징:

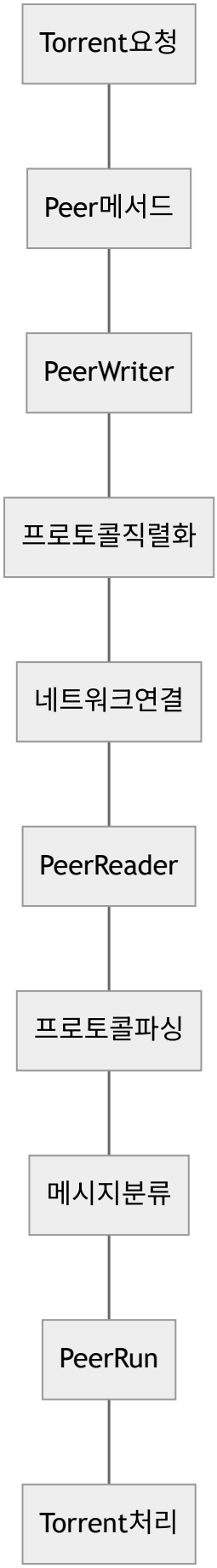
- **논블로킹 전달**: 새 연결을 채널로 즉시 전달
- **그레이스풀 종료**: 종료 신호 시 리스너 정리
- **에러 처리**: 연결 실패 시 로깅 후 계속 진행

5. 네트워킹 패키지 간 상호작용

5.1 연결 설정 과정


Acceptor
Session
Peer
PeerConn
PeerReader
PeerWriter
새 연결 전달
Peer 객체 생성
PeerConn 생성
PeerReader 시작
PeerWriter 시작
Run() 루프 시작
Acceptor
Session
Peer
PeerConn
PeerReader
PeerWriter

5.2 메시지 처리 흐름


Torrent요청
Peer메서드
PeerWriter
프로토콜직렬화
네트워크연결
PeerReader
프로토콜파싱
메시지분류
PeerRun
Torrent처리

5.3 성능 최적화 기법

1. 버퍼 풀링:

- `blockPool` 을 통한 메모리 재사용
- 피스 데이터 읽기 시 할당 최소화

2. 비동기 처리:

- Reader/Writer 분리로 전이중 통신
- 채널 기반 논블로킹 메시지 전달

3. Rate Limiting:

- `ratelimit.Bucket` 으로 대역폭 제어
- 업로드/다운로드 속도 독립 제한

4. 연결 관리:

- Keep-alive 메시지로 연결 유지
- 타임아웃 기반 비정상 연결 감지

6. 에러 처리 및 복구

6.1 네트워크 에러 처리

연결 끊김 감지:

```
// PeerReader에서 EOF 처리
if err == io.EOF {
return // 정상 종료
} else if err == io.ErrUnexpectedEOF {
return // 예상치 못한 종료
} else if _, ok := err.(*net.OpError); ok {
```

```
return  // 네트워크 오류
}
```

타임아웃 처리:

```
err = p.conn.SetReadDeadline(time.Now().Add(readTimeout))
if  nerr, ok := err.(net.Error);  ok && nerr.Timeout() {
// 타임아웃 시 재시도 또는 연결 종료
}
```

6.2 프로토콜 에러 처리

잘못된 메시지 크기:

```
if  rm.Length > MaxBlockSize {
err = &blockSizeError{
messageID: id,
got: rm.Length,
allowedMax: MaxBlockSize,
}
return
}
```

알 수 없는 메시지:

```
default:
p.log.Debugf("unhandled message type: %s", id)
_, err = io.CopyN(io.Discard, p.r, int64(length))
continue
```

7. 관련 문서

선행 지식

- 01-system-architecture.md - 네트워킹 계층의 전체 시스템 위치
- 02-data-flow-analysis.md - 네트워크 메시지 데이터 흐름

관련 주제

- 05-02-pex-protocol-analysis.md - PEX 프로토콜 메시지 처리
- 05-03-mse-encryption-analysis.md - 네트워크 암호화 통합
- 06-02-concurrency-analysis.md - 피어 고루틴 관리

후속 학습

- 03-01-torrent-lifecycle-process.md - 토렌트에서의 피어 연결 활용
- 06-03-error-handling-recovery-analysis.md - 네트워크 오류 처리

> **네비게이션:** 이전: 토렌트 라이프사이클 | 목차 | 다음: DHT 구현 분석

이 네트워킹 패키지들은 BitTorrent 프로토콜의 복잡한 네트워크 통신을 효율적이고 안정적으로 구현하여 Rain 클라이언트의 핵심 기능을 제공합니다.